SmartSure

Insurance Management System

Architecture · Database Schema · API Endpoints · Event Contracts

Angular 17 • ASP.NET Core • Ocelot Gateway
SQL Server • EF Core Code-First • JWT • RabbitMQ

1. System Architecture Overview

SmartSure is a microservices-based insurance management platform built on .NET 10 and Angular 17. All client requests originate from the Angular SPA and route exclusively through the Ocelot API Gateway, which centralises JWT validation, rate limiting, and service dispatch. Each microservice owns its own EF Core DbContext with an isolated SQL Server schema and communicates asynchronously with other services through RabbitMQ using the Outbox pattern.

1.1 Architecture Layers

Layer	Technology	Responsibility
Presentation	Angular 17 SPA	Policyholder & Admin dashboards, route guards, HTTP interceptors, lazy-loaded modules
API Gateway	Ocelot · .NET 10	Centralised JWT validation, routing, rate limiting, load balancing
Identity Service	ASP.NET Core + JWT + BCrypt	Authentication, user management, role assignment, token issuance
Policy Service	ASP.NET Core + EF Core	Insurance types/subtypes, policy lifecycle, premiums, payments
Claims Service	ASP.NET Core + EF Core	Claims lifecycle, document upload, status transitions, audit history
Admin / Reporting	ASP.NET Core + EF Core	Claim approvals, report generation, audit logging, admin operations
Messaging	RabbitMQ + MassTransit	Async domain events with Outbox pattern for exactly-once delivery
Database	SQL Server (per-service DbContext)	Isolated schema per microservice; cross-service refs are logical only

1.2 Request Flow

- User interacts with the Angular SPA (Policyholder or Admin dashboard)
- Angular attaches a JWT Bearer token and sends the request to the Ocelot Gateway
- Ocelot validates the JWT signature and expiry, then routes to the appropriate microservice
- The microservice processes the request using its own EF Core DbContext and returns a response
- For async operations (e.g. claim submitted), the microservice writes an OutboxMessage in the same DB transaction
- A background poller publishes the outbox event to RabbitMQ; the consuming service handles it independently

1.3 Security Model

- JWT tokens are issued by the Identity Service and verified by Ocelot on every inbound request
- Roles: Customer, Admin — enforced via [Authorize(Roles)] attributes on all protected controllers
- Angular route guards protect /customer/* and /admin/* routes at the client level
- Passwords are stored as BCrypt hashes in a dedicated Passwords table (never in the Users record)
- Cross-service data references use logical (non-FK) identifiers to maintain full service isolation

2. Architecture Diagram

The diagram below shows all five architecture layers and the flow of data between them — from the Angular SPA through the Ocelot Gateway, across the four .NET 10 microservices, through RabbitMQ, and out to external infrastructure.

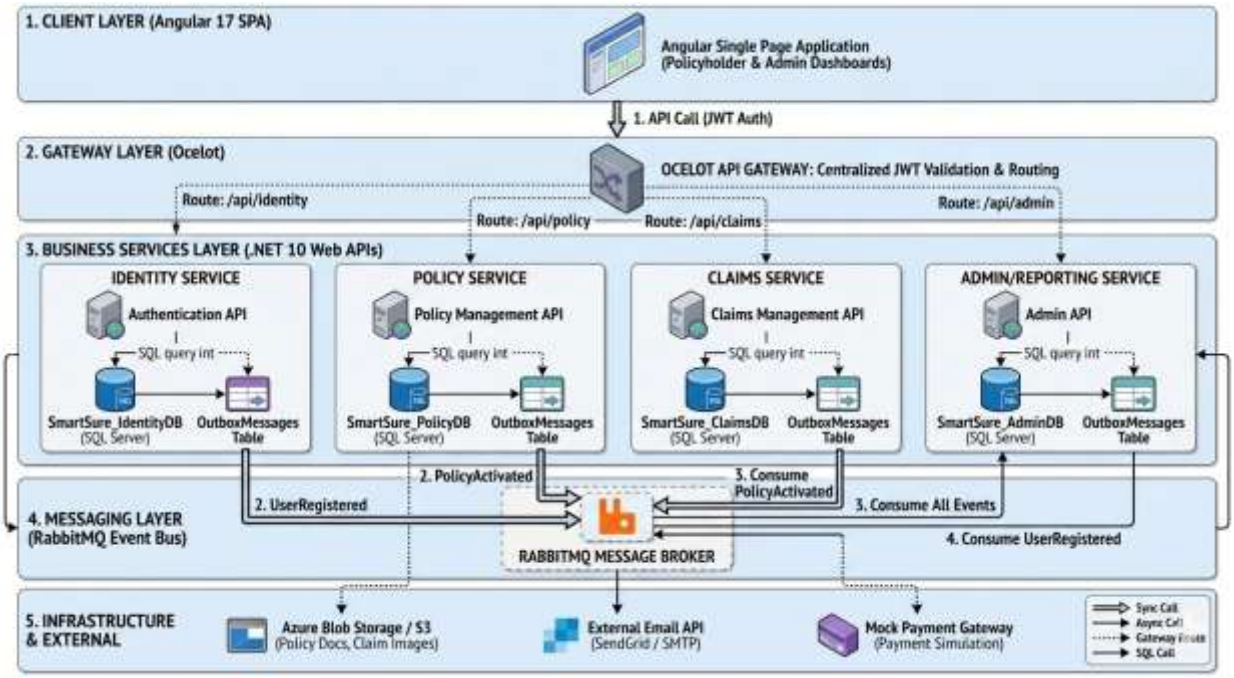

Figure 1 — SmartSure five-layer system architecture (Angular SPA → Ocelot → Microservices → RabbitMQ → Infrastructure)

2.1 Layer Descriptions

Layer 1 **Client Layer** · *Angular 17 SPA*

A single Angular 17 SPA serves both the Policyholder and Admin dashboards. Lazy-loaded feature modules keep the initial bundle small. HTTP interceptors automatically attach the JWT Bearer token to every outgoing API request.

Layer 2 **Gateway Layer** · *Ocelot API Gateway · .NET 10*

All external traffic enters through a single Ocelot gateway. Ocelot validates the JWT, then dispatches to the correct downstream service based on the route prefix.

Route Prefix	Downstream Service	Port
/api/identity	Identity Service	5001
/api/policy	Policy Management Service	5002
/api/claims	Claims Service	5003
/api/admin	Admin / Reporting Service	5004

Layer 3 **Business Services Layer** · *.NET 10 Web APIs*

Four independent ASP.NET Core Web APIs, each with its own EF Core DbContext, SQL Server schema, and Outbox table. Services are deployed independently and communicate either via the gateway (synchronous, client-initiated) or via RabbitMQ (asynchronous, event-driven).

Layer 4 Messaging Layer · *RabbitMQ · MassTransit · Outbox Pattern*

Asynchronous integration is handled by RabbitMQ with Topic exchanges and durable queues. MassTransit wraps the broker client. The Outbox pattern guarantees at-least-once delivery: events are written to an OutboxMessages table in the same DB transaction as the business operation, then published by a background worker.

Layer 5 Infrastructure & External · *Blob Storage · Email API · Payment Gateway*

Component	Purpose	Used By
Azure Blob Storage / S3	Store policy documents and claim images	Policy Service, Claims Service
External Email API (SendGrid / SMTP)	Send transactional notifications	Admin Service, Identity Service
Mock Payment Gateway	Simulate payment processing for premium flows	Policy Service

3. Database Schema

Each microservice owns its own EF Core DbContext and SQL Server schema. Cross-service references use logical (non-enforced) foreign keys marked "Logical Ref" — no cross-database FK constraints exist, preserving service independence. Every database includes an OutboxMessages table for the Outbox pattern.

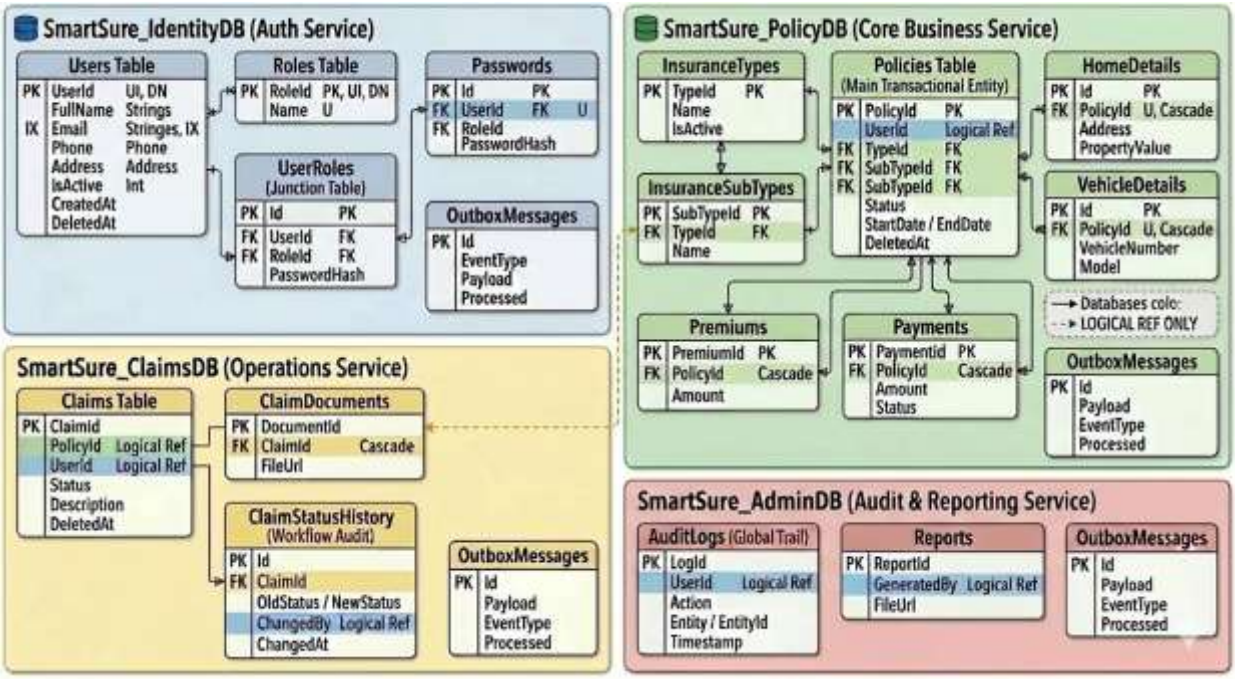

Figure 2 — SmartSure database schema across all four microservice databases

3.1 SmartSure_IdentityDB (Auth Service)

SmartSure_IdentityDB			
Table	**Type**	**Key Fields**	**Relationships**
Users	Main entity	PK: UserId · UI, DN | FullName, Email (IX), Phone, Address, IsActive, CreatedAt, DeletedAt	→ UserRoles, Passwords
Roles	Lookup	PK: RoleId · UI, DN | Name (U)	← UserRoles
UserRoles	Junction	PK: Id | FK: UserId, RoleId, PasswordHash	Users × Roles many-to-many
Passwords	Auth	PK: Id | FK: UserId (U) | RoleId, PasswordHash	→ Users (1:1)
OutboxMessages	Messaging	PK: Id | EventType, Payload, Processed	Outbox pattern

3.2 SmartSure_PolicyDB (Core Business Service)

SmartSure_PolicyDB			
Table	**Type**	**Key Fields**	**Relationships**
InsuranceTypes	Lookup	PK: TypeId | Name, IsActive	→ InsuranceSubTypes

InsuranceSubTypes	Lookup	PK: SubTypeId | FK: TypeId | Name	← InsuranceTypes
Policies	Main transactional	PK: PolicyId | UserId (Logical Ref) | FK: TypeId, SubTypeId | Status, StartDate, EndDate, DeletedAt	→ Premiums, Payments, HomeDetails, VehicleDetails
Premiums	Financial	PK: PremiumId | FK: PolicyId (Cascade) | Amount	← Policies
Payments	Financial	PK: PaymentId | FK: PolicyId (Cascade) | Amount, Status	← Policies
HomeDetails	Type-specific	PK: Id | FK: PolicyId (U, Cascade) | Address, PropertyValue	← Policies (home type)
VehicleDetails	Type-specific	PK: Id | FK: PolicyId (U, Cascade) | VehicleNumber, Model	← Policies (vehicle type)
OutboxMessages	Messaging	PK: Id | EventType, Payload, Processed	Outbox pattern

3.3 SmartSure_ClaimsDB (Operations Service)

SmartSure_ClaimsDB			
Table	**Type**	**Key Fields**	**Relationships**
Claims	Main entity	PK: ClaimId | PolicyId (Logical Ref) | UserId (Logical Ref) | Status, Description, DeletedAt	→ ClaimDocuments, ClaimStatusHistory
ClaimDocuments	Attachments	PK: DocumentId | FK: ClaimId (Cascade) | FileUrl	← Claims
ClaimStatusHistory	Workflow audit	PK: Id | FK: ClaimId | OldStatus, NewStatus | ChangedBy (Logical Ref) | ChangedAt	← Claims
OutboxMessages	Messaging	PK: Id | EventType, Payload, Processed	Outbox pattern

3.4 SmartSure_AdminDB (Audit & Reporting Service)

SmartSure_AdminDB			
Table	**Type**	**Key Fields**	**Relationships**
AuditLogs	Global trail	PK: LogId | UserId (Logical Ref) | Action, Entity, EntityId, Timestamp	Cross-service audit record
Reports	Generated output	PK: ReportId | GeneratedBy (Logical Ref) | FileUrl	→ Users (logical)
OutboxMessages	Messaging	PK: Id | EventType, Payload, Processed	Outbox pattern

4. Microservice Breakdown

4.1 Identity & Auth Service

Identity & Auth Service Port 5001 · /api/identity

Database	Owned Entities	Publishes	Consumes
SmartSure_IdentityDB	Users Roles UserRoles Passwords OutboxMessages	UserRegistered UserLoggedIn	— none —

4.2 Policy Management Service

Policy Management Service Port 5002 · /api/policy

Database	Owned Entities	Publishes	Consumes
SmartSure_PolicyDB	InsuranceTypes InsuranceSubTypes Policies Premiums Payments HomeDetails VehicleDetails OutboxMessages	PolicyActivated PolicyCancelled	UserRegistered (seed customer)

4.3 Claims Service

Claims Service Port 5003 · /api/claims

Database	Owned Entities	Publishes	Consumes
SmartSure_ClaimsDB	Claims ClaimDocuments ClaimStatusHistory OutboxMessages	ClaimSubmitted ClaimStatusChanged	PolicyActivated (validate eligibility)

4.4 Admin & Reporting Service

Admin & Reporting Service Port 5004 · /api/admin

Database	Owned Entities	Publishes	Consumes
SmartSure_AdminDB	AuditLogs Reports OutboxMessages	ClaimApproved ClaimRejected	ClaimSubmitted PolicyActivated UserRegistered

5. API Endpoints

All routes are exposed via the Ocelot Gateway. Total: 54 endpoints across 4 services. Auth levels: Public (no token), Bearer (any authenticated user), Admin (Admin role required).

5.1 Identity & Auth Service — 8 endpoints (/api/identity)

Method	Route	Description	Auth	Service
POST	/auth/register	Register a new customer account	Public	Identity
POST	/auth/login	Login and receive JWT token	Public	Identity
POST	/auth/logout	Logout and invalidate token	Bearer	Identity
GET	/auth/me	Get current user profile	Bearer	Identity
PUT	/auth/me	Update name, phone, address	Bearer	Identity
PUT	/auth/change-password	Change password (BCrypt)	Bearer	Identity
GET	/auth/users	List all users	Admin	Identity
PUT	/auth/users/{userId}/roles	Assign role to a user	Admin	Identity

5.2 Policy Management Service — 21 endpoints (/api/policy)

Method	Route	Description	Auth	Service
GET	/insurance-types	List all insurance types	Bearer	Policy
GET	/insurance-types/{typeId}	Get a single insurance type	Bearer	Policy
POST	/insurance-types	Create an insurance type	Admin	Policy
PUT	/insurance-types/{typeId}	Update an insurance type	Admin	Policy
GET	/insurance-types/{typeId}/subtypes	List subtypes under a type	Bearer	Policy
POST	/insurance-subtypes	Create a subtype	Admin	Policy
PUT	/insurance-subtypes/{subTypeId}	Update a subtype	Admin	Policy
GET	/policies	List my policies	Bearer	Policy
GET	/policies/{policyId}	Get a single policy	Bearer	Policy
POST	/policies	Buy a new policy (wizard submit)	Bearer	Policy
PUT	/policies/{policyId}/cancel	Cancel a policy	Admin	Policy
GET	/policies/{policyId}/details	Get policy details document	Bearer	Policy
POST	/policies/{policyId}/details	Save policy details	Bearer	Policy
PUT	/policies/{policyId}/details	Update policy details	Bearer	Policy
GET	/policies/{policyId}/premium	Get premium amount for a policy	Bearer	Policy
POST	/policies/{policyId}/payments	Record a payment	Bearer	Policy
GET	/policies/{policyId}/payments	List payments for a policy	Bearer	Policy
GET	/home-details/{policyId}	Get home insurance details	Bearer	Policy

Method	Route	Description	Auth	Service
POST	/home-details	Save home insurance details	Bearer	Policy
GET	/vehicle-details/{policyId}	Get vehicle insurance details	Bearer	Policy
POST	/vehicle-details	Save vehicle insurance details	Bearer	Policy

5.3 Claims Service — 10 endpoints (/api/claims)

Method	Route	Description	Auth	Service
GET	/claims	List my claims	Bearer	Claims
GET	/claims/{claimId}	Get a single claim	Bearer	Claims
POST	/claims	Initiate a new claim (wizard)	Bearer	Claims
PUT	/claims/{claimId}	Update a draft claim	Bearer	Claims
PUT	/claims/{claimId}/submit	Submit claim for review	Bearer	Claims
GET	/claims/{claimId}/documents	List documents for a claim	Bearer	Claims
POST	/claims/{claimId}/documents	Upload a claim document	Bearer	Claims
DELETE	/claims/{claimId}/documents/{docId}	Delete a claim document	Bearer	Claims
GET	/claims/{claimId}/history	Get claim status history	Bearer	Claims
GET	/claims/by-policy/{policyId}	Get claims for a specific policy	Bearer	Claims

5.4 Admin & Reporting Service — 14 endpoints (/api/admin)

Method	Route	Description	Auth	Service
GET	/admin/claims	List all claims	Admin	Admin
GET	/admin/claims/{claimId}	View claim with docs + history	Admin	Admin
PUT	/admin/claims/{claimId}/review	Set claim to Under Review	Admin	Admin
PUT	/admin/claims/{claimId}/approve	Approve a claim	Admin	Admin
PUT	/admin/claims/{claimId}/reject	Reject a claim with reason	Admin	Admin
GET	/admin/policies	List all policies	Admin	Admin
GET	/admin/users	List all users	Admin	Admin
GET	/admin/users/{userId}	Get user detail	Admin	Admin
DELETE	/admin/users/{userId}	Deactivate a user account	Admin	Admin
GET	/admin/dashboard	Get dashboard stats and KPIs	Admin	Admin
GET	/admin/reports	List all generated reports	Admin	Admin
POST	/admin/reports	Generate a new report	Admin	Admin
GET	/admin/reports/{reportId}	Download / view a report	Admin	Admin
GET	/admin/audit-logs	List audit logs (paginated)	Admin	Admin

5.5 Endpoint Count Summary

Service	GET	POST	PUT	DELETE	Total
Identity & Auth	3	3	2	0	8
Policy Management	11	7	3	0	21
Claims Service	5	2	2	1	10
Admin & Reporting	7	1	3	1	12
TOTAL	26	13	10	2	51

6. RabbitMQ Event Contracts

Exchange type: Topic · Durable: true · Message library: MassTransit (wraps RabbitMQ). All events use the Outbox pattern — written atomically with the business transaction and published by a background poller.

Event	Publisher	Consumer(s)	Payload / Purpose
UserRegistered	Identity	Policy, Admin	UserId, Email, FullName — seed customer profile in Policy & Admin
PolicyActivated	Policy	Claims, Admin	PolicyId, UserId, TypeId, SubTypeId — unlock claim eligibility
PolicyCancelled	Policy	Admin	PolicyId, UserId, Reason — audit trail entry
ClaimSubmitted	Claims	Admin	ClaimId, PolicyId, UserId, Description — trigger admin review queue
ClaimStatusChanged	Claims	Admin	ClaimId, OldStatus, NewStatus, ChangedBy — status audit log
ClaimApproved	Admin	Claims	ClaimId, AdminId, Notes — finalise claim record
ClaimRejected	Admin	Claims	ClaimId, AdminId, Reason — close with rejection reason

7. Folder Structure

The solution is organised as a monorepo under SmartSure/ with one project folder per deployable unit.

7.1 Solution Root

Folder / File	Description
SmartSure.Gateway/	Ocelot API Gateway — routing configuration, JWT validation middleware
SmartSure.IdentityService/	Identity & Auth microservice — users, roles, JWT issuance
SmartSure.PolicyService/	Policy Management microservice — policies, premiums, payments
SmartSure.ClaimsService/	Claims microservice — claims lifecycle, documents, status history
SmartSure.AdminService/	Admin & Reporting microservice — approvals, audit logs, reports
SmartSure.Shared/	Shared library — event contracts, DTOs, status constants
SmartSure.Angular/	Angular 17 SPA — policyholder and admin frontend
SmartSure.sln	Visual Studio solution file
docker-compose.yml	Multi-container orchestration for local development
README.md	Project overview, setup guide, and architecture notes

7.2 Per-Service Internal Structure

Each microservice follows the same internal folder pattern:

Folder	Contents
Controllers/	HTTP controllers — one per domain aggregate (e.g. ClaimsController, PoliciesController)
Services/	Business logic — interfaces (IClaimService) + implementations (ClaimService)
Models/	EF Core entity classes mapped to database tables
DTOs/	Request and response data transfer objects (Create, Update, Response DTOs)
Data/	DbContext subclass + EF Core Migrations/ subfolder
Events/	RabbitMQ event classes published by this service (e.g. ClaimSubmittedEvent)
Consumers/	MassTransit consumers for events from other services
appsettings.json	DB connection strings, JWT config, RabbitMQ connection URL

7.3 Angular Frontend Structure (SmartSure.Angular/src/app/)

Module / Folder	Contents
core/	CoreModule: layout components, auth/error interceptors, route guards, shared services
shared/	SharedModule: status badges, data tables, modal wrappers, stepper wizard component
auth/	AuthModule: login and register components (public routes)
customer/	CustomerModule (lazy): dashboard, buy-policy wizard, policy detail, document upload
claims/	ClaimsModule (lazy): initiate claim wizard, claim list, claim status timeline
admin/	AdminModule (lazy, role-guarded): claim review, policy management, user management

Module / Folder	Contents
reports/	ReportsModule (lazy): report list and generate report form
models/	TypeScript interfaces mirroring all backend DTOs
services/	Angular HTTP services: policy.service.ts, claims.service.ts, admin.service.ts
environments/	environment.ts and environment.prod.ts — API base URL configuration